

Tensegrity Rover Simulator: Architecture, Implementation, and Future Trajectories

Student Name
Department of Engineering/Computer Science
University Name
Email: student@university.edu

Abstract— This paper presents the architecture, current technology stack, and future trajectory of a browser-based 3D physics simulation designed for a six-strut, 24-cable tensegrity rover. The project serves as an accessible, high-performance digital twin for soft robotics, integrating advanced MATLAB-based locomotion controllers (LQR, iLQR, QP-MPC, and Neural) directly into a 500 Hz JavaScript rigid-body solver. Through iterative payload isolation tests and dynamic terrain generation, this simulator establishes a robust framework for extraterrestrial robotics research. Future expansions aim to leverage WebAssembly (Wasm) and WebGPU for compute-heavy parallelizations, as well as live TensorFlow.js integration and Robot Operating System (ROS) connectivity for hardware-in-the-loop (HIL) applications.

Index Terms—Tensegrity Robotics, Browser Simulation, Model Predictive Control, WebAssembly, Soft Robotics

I. INTRODUCTION

The exploration of unpredictable environments, such as the Martian surface, requires highly adaptable and robust locomotion systems. Tensegrity rovers—structures composed of isolated rigid struts suspended within a continuous network of tensioned cables—offer unparalleled impact absorption and structural resilience.

This repository presents a high-fidelity, real-time 3D simulation of such a rover. Designed to run natively in a web browser without the overhead of massive game engines, the project relies on custom numerical integrators and advanced control theory to accurately model the physical interactions of a six-bar tensegrity structure against rough terrains. The core objective of the repository is to provide a platform for validating advanced locomotion controllers and testing the shock-absorption capabilities of a suspended inner payload.

II. CURRENT TECHNOLOGICAL FRAMEWORK

The architecture of the simulator is built strictly on highly accessible and optimized web technologies, maximizing deployment reach while maintaining computational rigor.

A. Core Simulation Engine

The simulation operates entirely on Vanilla JavaScript (ES6+), circumventing the need for external frameworks. The physics engine performs numerical integration (using Semi-implicit Euler) at a fixed step rate of 500 Hz. It handles rigid-body dynamics, Hertzian contact forces, Coulomb friction, and non-linear spring-damper cable physics natively.

B. Additive Controller Suite

A primary feature of the simulator is its Additive Drive Controller Suite, which ports complex MATLAB models directly into the JavaScript environment. These include:

- Linear Quadratic Regulators (LQR)
- Iterative LQR (iLQR) with Minimax variations for disturbance rejection
- Quadratic Programming Model Predictive Control (QP-MPC) for strict adherence to actuator constraints
- Neural Network geometry heuristics for $O(1)$ inference speed

C. Environment and Rendering

The simulation visually renders multi-octave procedural Martian terrains using WebGL (via Three.js). Environments range from flat baseline tracks to 1 km² open-world expeditions featuring over 40,000 uniquely instanced obstacles. Rendering is optimized to run alongside the heavy physics thread seamlessly.

III. EXPERIMENTAL VERIFICATION

The platform inherently supports robust experimental validation, specifically demonstrated through the "Level 10" Central Payload Shock test. In this scenario, a 1.6 kg central payload is structurally isolated using the tensegrity network. The simulator dynamically tracks the proper acceleration (G-force) experienced by the core as the rover traverses 10 staggered monolithic spheres. The results continuously demonstrate that the complex network of tensioned cables absorbs the impacts, keeping the internal core below the strict 1.5 G structural threshold despite extreme external cage deformation.

IV. FUTURE TRAJECTORIES

As the fidelity of the simulation and the complexity of the control models scale, the underlying technology stack must evolve. The future roadmap includes three critical architectural upgrades.

A. WebAssembly and GPU Acceleration

To overcome the limits of the JavaScript V8 engine, the core 500 Hz physics loops and matrix solvers will be rewritten in Rust or C++ and compiled to WebAssembly (Wasm). Furthermore, WebGPU will be implemented for compute shaders, allowing massive parallelization of collision detection across tens of thousands of procedural ground elements.

B. Live Artificial Intelligence

Rather than relying on pre-computed weight matrices exported from MATLAB, the framework will integrate TensorFlow.js. This will allow the rover to utilize

Reinforcement Learning (RL) agents that train dynamically inside the browser, adapting to novel terrains in real-time.

C. ROS Integration

The ultimate goal is to bridge the digital twin with physical hardware. By implementing WebSockets to a Python backend running the Robot Operating System (ROS), the browser application will send validated, simulated control trajectories directly to a physical tensegrity rover in the real world.

V. CONCLUSION

The Tensegrity Rover Simulator stands as a comprehensive digital twin for soft robotics research. By currently leveraging highly optimized JavaScript for complex MATLAB control execution, and planning for WebAssembly and machine learning integrations, the repository offers a scalable, powerful, and accessible tool for advancing extraterrestrial robotics.